UrbanCase

Documentación técnica del proyecto

Punto de venta (POS) para accesorios de celular

Contenido: arquitectura general, organización de carpetas, explicación del backend y el frontend, modelo de datos, seguridad, despliegue y flujos principales del sistema.

1. Resumen del proyecto

UrbanCase es un sistema de punto de venta (POS) para una tienda de accesorios de celular con varias sucursales. Permite llevar inventario por sucursal, registrar ventas en efectivo o con tarjeta, emitir tickets, dar seguimiento a clientes y proveedores, calcular comisiones por ventas consignadas y consultar reportes/indicadores del negocio desde un panel central (Home).

El proyecto está dividido en dos partes claramente separadas: una interfaz web (carpeta **public/**) y una API del lado del servidor (carpeta **server/**) que expone endpoints REST y habla directamente con la base de datos.

1.1 Tecnologías utilizadas

Capa	Tecnología	Uso en el proyecto
Frontend	HTML + CSS + JavaScript (sin frameworks)	Interfaz completa como aplicación de una sola página (SPA)
Frontend	Chart.js	Gráfica de ventas de los últimos 7 días en el Home
Backend	Node.js + Express 5	Servidor HTTP y API REST
Backend	pg (node-postgres)	Conexión y consultas SQL directas a PostgreSQL (sin ORM)
Backend	bcryptjs	Hash de contraseñas de usuarios
Backend	pdfkit	Generación de tickets de venta en PDF
Backend	sharp	Generación de miniaturas de imágenes de producto
Base de datos	PostgreSQL (Supabase)	Almacenamiento de todo el negocio
Hosting	Railway	Despliegue del servidor Node.js (24/7)
Hosting	Supabase	Base de datos PostgreSQL administrada

1.2 Roles de usuario

Rol	Alcance
Dueño	Acceso completo; único rol que puede fijar un precio libre en el carrito de venta
Developer	Acceso completo; administración técnica del sistema
Admin	Acceso completo a los módulos de su(s) sucursal(es); genera comisión
Vendedor	Solo Inventario y carrito/cobro de su sucursal; no administra catálogo ni usuarios

2. Arquitectura general

La aplicación sigue una arquitectura cliente-servidor clásica de tres capas: interfaz web, API REST y base de datos relacional.

```
Navegador (public/)
| fetch() con JSON + token Bearer
v
API Express (server/index.js)
| middlewares: seguridad, CORS, autenticacion, permisos
v
Rutas REST (server/routes/*.js)
| logica de negocio reutilizable
v
Servicios (server/services/*.js)
| SQL directo via 'pg'
v
PostgreSQL (Supabase, region us-west2)
```

El servidor Express corre de forma continua en **Railway** (región us-west2) y se conecta a una base de datos **PostgreSQL administrada por Supabase** a través del pooler de sesión. No existe un ORM: todas las consultas son SQL escrito a mano usando la librería **pg**, lo que da control total sobre el rendimiento de cada consulta.

La autenticación es por token: al iniciar sesión el servidor crea un token aleatorio y lo guarda en memoria junto con los datos del usuario (sin usar la base de datos ni JWT). El cliente lo manda de vuelta en cada petición con el encabezado **Authorization: Bearer <token>**.

3. Organización de carpetas

```
UrbanCase/
|-- public/ Interfaz web (lo que se sirve al navegador)
| |-- index.html Aplicacion principal (Home, Inventario, Ventas, ...)
| |-- login.html Pantalla de inicio de sesion
| |-- ticket.html Vista de ticket de venta
| |-- css/styles.css Estilos (tema claro/oscuro)
| |-- js/app.js Logica principal de la SPA
| |-- js/home-dashboards.js KPIs y graficas del Home
| |-- js/theme.js Cambio de tema claro/oscuro
| |-- js/uc-datepicker.js Selector de fechas a la medida
| `-- img/ Logos e iconos
|
|-- server/ API y acceso a datos
| |-- index.js Punto de entrada: monta middlewares y rutas
| |-- config/db.js Pool de conexion a PostgreSQL
| |-- middleware/ Seguridad, sesiones y permisos (ver seccion 5)
| |-- routes/ Un archivo por recurso de la API (ver seccion 4.2)
| |-- services/ Logica de negocio reutilizable (ver seccion 4.3)
| |-- sql/ schema.sql + migraciones incrementales
| |-- scripts/ Scripts de mantenimiento de un solo uso
| `-- tickets/ PDFs de tickets generados en tiempo de ejecucion
|
|-- docs/ Historias de uso y notas de seguridad
|-- respaldos/ Copias locales de datos (no se suben a git)
|-- package.json Scripts npm start / npm run server
`-- railway.toml Configuracion de despliegue en Railway
```

La separación entre **public/** y **server/** es intencional: todo lo que está dentro de **public/** es estático y se manda tal cual al navegador; todo lo que está en **server/** corre únicamente en el servidor y nunca se expone (de hecho, un middleware — *static-guard* — bloquea explícitamente cualquier intento de pedir archivos de esa carpeta por HTTP).

4. Backend (server/)

4.1 Punto de entrada: index.js

Crea la aplicación de Express, aplica los middlewares globales en orden (cabeceras de seguridad, CORS, parseo de JSON), expone **/api/health** para monitoreo, monta cada grupo de rutas bajo su prefijo (**/api/auth**, **/api/productos**, **/api/ventas**, etc.) y al final sirve los archivos estáticos de **public/**. También arranca un ping periódico cada 24 horas a la base de datos para que Supabase nunca la vea inactiva y la pause.

4.2 Rutas (server/routes/)

Cada archivo expone los endpoints REST de un recurso del negocio:

Archivo	Responsabilidad
auth.js	Login, logout y perfil de la sesión actual
productos.js	Catálogo: alta, edición, baja, listado, imágenes/miniaturas, restock rápido
productos-consignados.js	Catálogo de productos en consignación
ventas.js	Registrar ventas, devoluciones, corte de caja
reportes.js	KPIs, gráfica de 7 días, top de productos, comparativa de sucursales
clientes.js	Alta/edición de clientes y precios especiales por cliente
proveedores.js	Catálogo de proveedores
usuarios.js	Alta/edición de usuarios (solo acceso completo)
sucursales.js	Alta/edición de sucursales
inventario-favoritos.js	Productos marcados como favoritos por cada usuario

4.3 Servicios (server/services/)

Contienen la lógica de negocio que varias rutas necesitan compartir, separada de la capa HTTP:

- **venta-core.js**: valida precios y arma las líneas de una venta (con bloqueo de fila sobre el stock)
- **venta-tarjeta.js** / **mp-point-order.js** / **mp-terminal-setup.js**: cobro con tarjeta a través de una terminal Mercado Pago Point
- **devolucion.js**: lógica de devolución de una venta
- **comisiones.js**: cálculo de comisión por ventas consignadas
- **inventario-movimientos.js**: historial de entradas/salidas de stock
- **productos-historial.js**: bitácora de alta/edición/baja de productos
- **cliente-precios.js**: precios especiales por cliente
- **ticket-contenido.js**, **ticket-emision.js**, **ticket-pdf.js**, **ticket-point.js**: construcción y emisión del ticket de venta (PDF e impresión en la terminal)

- **verificar-dueno.js**: confirma la contraseña del dueño para acciones sensibles (p. ej. borrar una sucursal)

4.4 Base de datos (server/sql/ y server/scripts/)

schema.sql define el esquema base; cada cambio posterior vive en su propio archivo **migration_*.sql**, con un script en **server/scripts/** que lo aplica de forma segura (usa siempre *IF NOT EXISTS*, por lo que correrlo más de una vez no causa error). **server/scripts/** también incluye utilidades que no forman parte de la API: respaldo de datos, copiado de productos entre sucursales, configuración inicial de Supabase y un script para dejar el ambiente de producción limpio antes de un corte real.

Tablas principales

Tabla	Contenido
usuarios	Cuentas, rol y sucursal asignada
sucursales	Locales de la tienda
productos / productos_consignados	Catálogo propio y en consignación, con stock por sucursal
ventas / venta_detalle	Encabezado y líneas de cada venta
devoluciones	Ventas devueltas
clientes / proveedores	Directorios de negocio
cliente_precios	Precio especial de un producto para un cliente
inventario_movimientos / inventario_traslados	Historial de stock y traslados entre sucursales
inventario_favoritos	Productos favoritos por usuario
productos_historial	Bitácora de cambios al catálogo

5. Seguridad (server/middleware/)

Middleware	Qué hace
auth.js	Valida el token recibido contra las sesiones guardadas en memoria (expiran a las 12 h)
rbac.js / roles.js	Control de acceso por rol: qué roles cuentan como «acceso completo»
login-limiter.js	Limita a 10 intentos de login por IP+usuario cada 15 minutos
security-headers.js	Cabeceras HTTP básicas de seguridad (X-Frame-Options, nosniff, etc.)
static-guard.js	Bloquea por HTTP cualquier ruta que empiece con /server, archivos .env, .sql o .md
errors.js	Oculta el detalle técnico de los errores cuando el servidor corre en producción

Las contraseñas de usuario se guardan con **bcrypt** (nunca en texto plano). La cadena de conexión a la base de datos y cualquier credencial viven únicamente en **server/.env**, un archivo que nunca se sube al repositorio (está en **.gitignore**) y solo existe en el servidor de despliegue.

6. Frontend (public/)

La interfaz es una aplicación de una sola página: **index.html** contiene todas las secciones (Home, Inventario, Ventas, Clientes y Proveedores, Gestión) como bloques que se muestran u ocultan desde JavaScript, sin recargar la página. No se usa React, Vue ni ningún framework: todo el manejo del DOM es JavaScript puro.

Archivo	Responsabilidad
index.html	Estructura de toda la aplicación (navbar + las distintas vistas)
login.html	Pantalla de acceso, independiente de index.html
ticket.html	Vista imprimible de un ticket de venta
js/app.js	Núcleo de la aplicación: inventario, carrito, cobro, clientes/proveedores, gestión, modales
js/home-dashboards.js	Carga y pinta los KPIs, la gráfica de 7 días, el top de productos y la comparativa de sucursales
js/theme.js	Alterna y recuerda el tema claro/oscuro
js/uc-datepicker.js	Selector de fechas propio usado en los filtros de reportes
css/styles.css	Todos los estilos, con variables por tema (claro/oscuro)

Las imágenes de producto se guardan como texto (base64) directamente en la base de datos. Para que el inventario cargue rápido, cada producto tiene dos versiones de su imagen: una miniatura pequeña (≈420px) que viaja con la lista completa de productos, y la imagen original, que solo se pide al ampliarla.

7. Despliegue

Servicio	Rol
Railway	Ejecuta el servidor Node.js de forma continua (región us-west2, EE. UU.)
Supabase	Base de datos PostgreSQL administrada (plan gratuito)
GitHub	Repositorio de código; cada push a la rama main dispara un redeploy automático en Railway

El plan gratuito de Supabase pausa un proyecto tras varios días sin actividad; para evitarlo, el propio servidor hace una consulta mínima a la base cada 24 horas mientras está en línea.

8. Flujos principales del sistema

8.1 Inicio de sesión

El usuario envía usuario y contraseña a **POST /api/auth/login**. El servidor compara la contraseña con el hash guardado (bcrypt); si coincide, crea una sesión en memoria con un token aleatorio y una expiración de 12 horas, y lo devuelve al cliente. El cliente guarda ese token y lo manda en cada petición posterior.

8.2 Una venta en el POS

- Se agregan productos al carrito desde Inventario (respetando el precio mínimo/máximo permitido para cada uno)
- Al cobrar, se elige método de pago: efectivo o tarjeta (vía terminal Mercado Pago Point)
- El servidor valida stock con bloqueo de fila, descuenta el inventario y registra la venta y sus líneas en una sola transacción
- Se genera el ticket en PDF y, si corresponde, se manda a imprimir en la terminal física

8.3 Dashboards del Home

Al entrar a Home, o al cambiar de periodo (hoy/semanal/mensual/anual) o de sucursal, el frontend pide en paralelo cuatro reportes (KPIs, gráfica de 7 días, top de productos y comparativa de sucursales). Cada consulta está escrita para aprovechar los índices de la base de datos y evitar transferir imágenes de tamaño completo, de forma que el panel responda en milisegundos incluso con el rango «anual».

8.4 Alta de un producto con imagen

Al guardar un producto con foto, el navegador la reduce a un tamaño razonable antes de enviarla; el servidor genera además una miniatura más pequeña con la librería **sharp** y guarda ambas versiones. La lista de inventario solo descarga las miniaturas, por lo que cargar decenas de productos con imagen es rápido.